

Talia and the Tollkeepers' Promise

Solution editorial

CITS3001 – Kruskal's Algorithm

Model and key insight. Given an undirected multigraph with nonnegative tolls, start at vertex 1, visit every vertex, and return. Each distinct road is charged only on its first use. Minimise total payment, or report **IMPOSSIBLE**. This is a minimum spanning tree (MST) problem. We use Kruskal's algorithm and disjoint-set union (DSU) to construct an MST efficiently.

Why a spanning tree solves a journey problem. The distinct roads used by any valid journey form a connected spanning subgraph. It contains a spanning tree of no greater cost because all tolls are nonnegative. Thus every journey costs at least the MST weight. Conversely, walking along a spanning tree, exploring its branches and returning along the same roads, visits every vertex and returns to 1. Each tree edge is paid for once, so an MST realises that lower bound. Repeated crossings do not double the cost. If the graph is disconnected, no valid journey exists. For $n = 1$, the empty journey costs zero.

Algorithm and data structures. Store all roads in a list of triples (w, u, v) and sort by nondecreasing toll. DSU stores **parent** and **size** arrays. **FIND** uses iterative path compression; a successful union attaches the smaller component to the larger one. Distinct roads remain distinct, including parallel roads. Ties may be processed in any order.

```
if n = 1: return 0
sort roads by increasing toll
parent[v] = v and size[v] = 1 for every vertex v
total = 0; chosen = 0
for each (w, u, v) in roads:
    a = Find(u); b = Find(v)
    if a = b: continue
    if size[a] < size[b]: swap(a, b)
    parent[b] = a; size[a] += size[b]
    total += w; chosen += 1
    if chosen = n - 1: return total
return IMPOSSIBLE
```

Why Kruskal's choices are correct. Assume the selected forest F is contained in some MST T , initially true for the empty forest. For the next selected edge $e = (u, v)$, let S be the current DSU component containing u . Edge e is a cheapest edge crossing $(S, V \setminus S)$: any earlier crossing edge would already have merged its endpoints. If $e \notin T$, adding e to T creates a cycle containing an edge f crossing the same cut. Such an f is not in F , and $w(e) \leq w(f)$. Replacing f by e preserves a spanning tree and does not increase cost, so an MST still contains the enlarged forest. After $n - 1$ unions it is an MST. If edges run out earlier, the original graph is disconnected.

Complexity and limits. Initialisation and input take $O(n + m)$ time. Sorting takes $O(m \log \max(2, m))$. At most $2m$ finds and $n - 1$ unions take $O((n + m)\alpha(n))$ amortised time with DSU, where α is the inverse Ackermann function. Overall time is $O(n + m \log \max(2, m) + m\alpha(n))$, and space is $O(n + m)$ including the edge list, DSU arrays, and sorting workspace. The n term matters for sparse or disconnected inputs. Local runs at the input limits finish within 5 seconds. The memory limit is 512 MiB. Final judge calibration remains necessary. A finite answer is at most $(n - 1)10^9 \leq 199\,999\,000\,000\,000$. Python integers handle this exactly.

Alternatives and pitfalls. Prim is also correct and supplies an independent reference solution. Dijkstra minimises individual path lengths, not the total cost of unlocking a shared network. Simply taking the cheapest $n - 1$ roads can form a cycle and miss a necessary bridge. Full-array component relabelling scans all n vertices after each successful merge. A connected graph needs $n - 1$ merges, so these scans alone take $O(n^2)$ time. Repeatedly searching the selected forest for each of up to m candidate roads can inspect $O(n)$ vertices per search, giving $O(mn)$ time in the worst case. Both methods also sort the roads. At these input limits, efficient DSU avoids the repeated scans.

AI-use declaration. This work is done independently.